CURSO DE CIÊNCIA DE DADOS • AULA 02

Redes Neurais, Embeddings e LLMs

Do neurônio artificial à linguagem contextual.



PERCEPTRON → REDES NEURAIS → REPRESENTAÇÃO DE TEXTO → TRANSFORMERS

Ponte com Regressão Logística

O princípio de combinar entradas com pesos.

Regressão logística

Entradas → combinação ponderada → escore → decisão.

Rede neural

Várias unidades calculam, transformam e enviam sinais umas às outras.

MAIS FLEXIBILIDADE

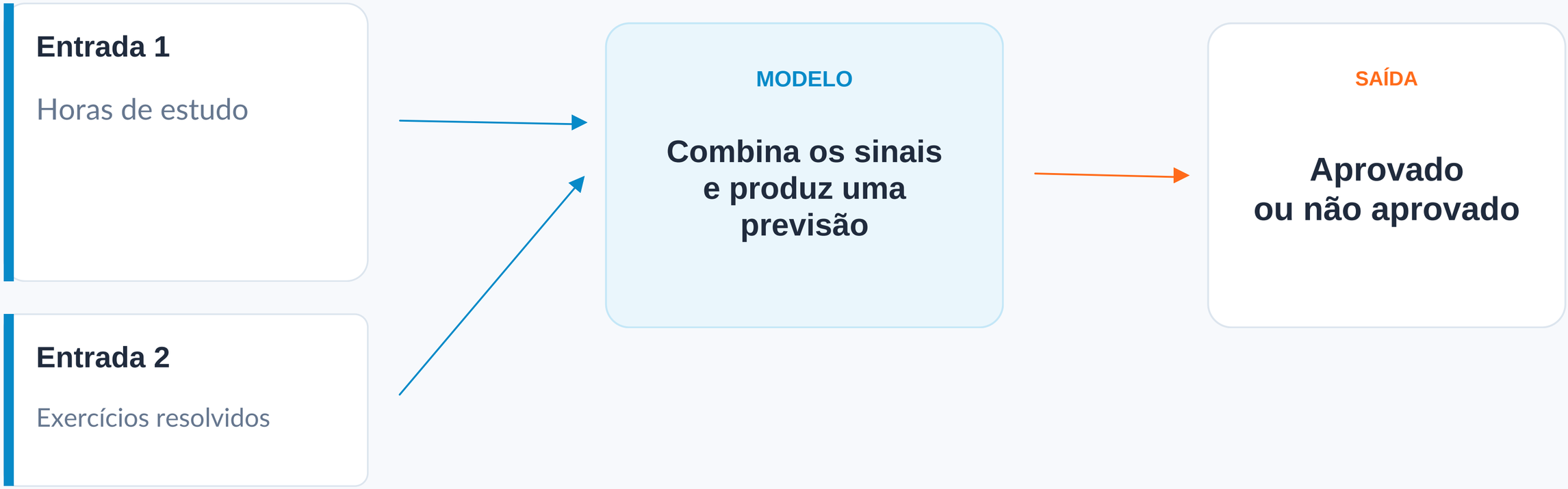
A rede constrói combinações intermediárias antes de produzir a resposta.

A mudança principal

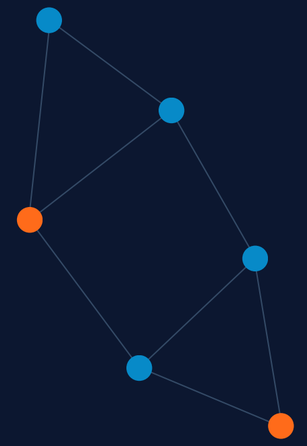
Ao invés de uma única “regressão”, encadearemos várias.

Exemplo: Aprovação numa Prova

Vamos pegar um exemplo de um modelo que prevê se o aluno passou numa prova.



Como uma rede neural toma decisões



Do cálculo de um neurônio à combinação de várias camadas.

ENTRADAS

Características do exemplo

PESOS + ATIVAÇÃO

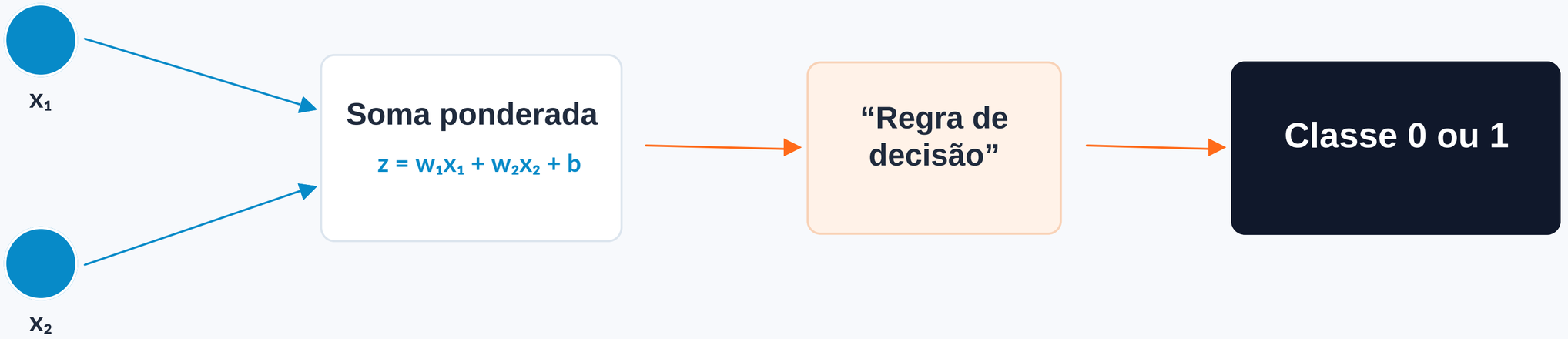
Transformar sinais

SAÍDA

Produzir uma decisão

Perceptron

O perceptron recebe números, combina-os e aplica uma regra simples para escolher uma classe.



Pesos (w)

Controlam o quanto cada entrada contribui.

Viés (b)

Desloca a regra de decisão, mesmo quando as entradas são zero.

Como o perceptron toma uma decisão?

Exemplo numérico.

EXEMPLO

x_1 = 3 horas de estudo
x_2 = 4 exercícios resolvidos

$w_1 = 0{,}8$ $w_2 = 0{,}5$ $b = -3$

$z = 0{,}8 \cdot 3 + 0{,}5 \cdot 4 - 3 = 1{,}4$

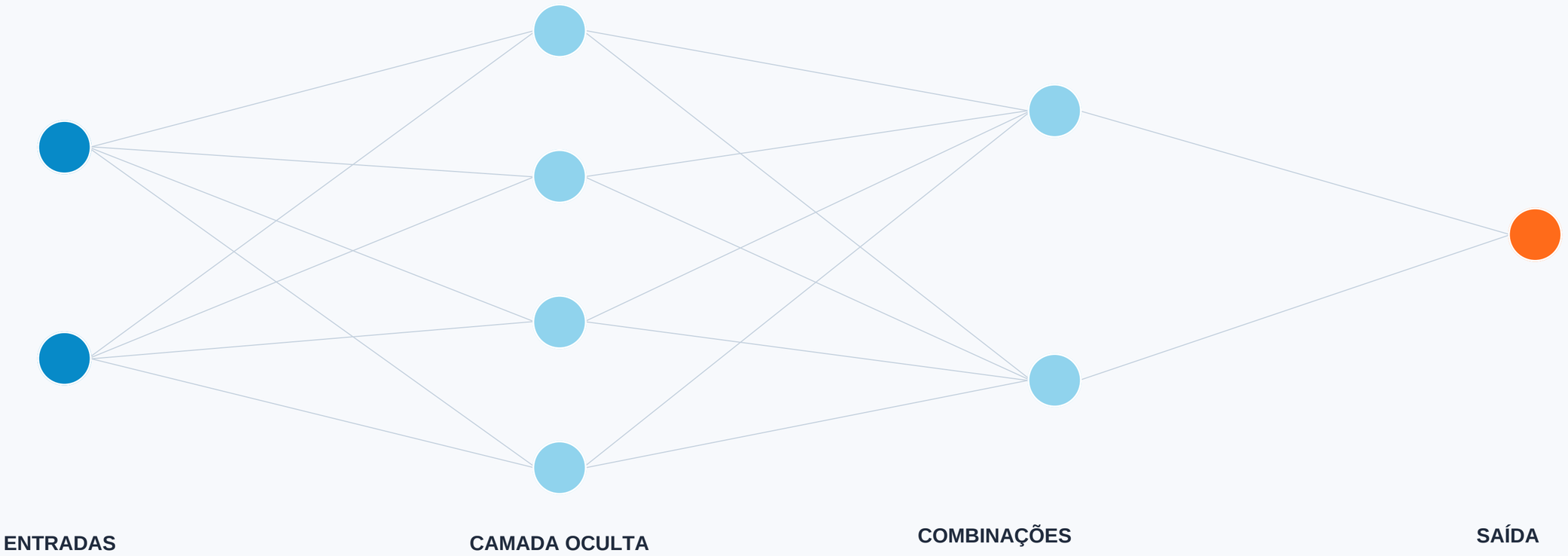
REGRA DE LIMIAR

Se $z > 0$ → classe 1
Se $z \leq 0$ → classe 0

Aqui: $z = 1{,}4$ → classe 1

De um neurônio para vários neurônios

Um percepetron é um neurônio, vários perceptrons podem formar uma rede neural.



A rede neural é um conjunto de várias “regressões” conectadas.

Camada de entrada: o que chega ao modelo

Cada coluna representa uma característica disponível para fazer a previsão.

EXEMPLO DE UMA OBSERVAÇÃO

$x_1 = 3$

Horas de estudo

$x_2 = 4$

Exercícios resolvidos



TENSOR DE ENTRADA

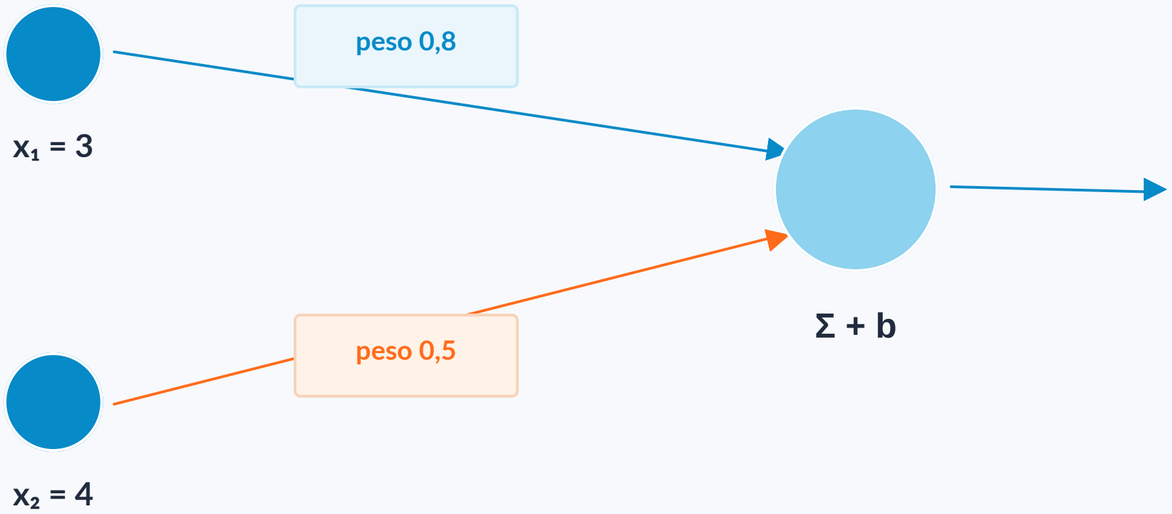
[[3, 4]]

1 linha = 1 pessoa
2 colunas = 2 características

Ao trabalhar em lotes, o formato é [quantidade de exemplos, quantidade de características].

Pesos: quanto cada entrada contribui

Cada neurônio possui um peso para cada entrada recebida.



Soma ponderada

$z = 0{,}8 \cdot 3 + 0{,}5 \cdot 4 + b$

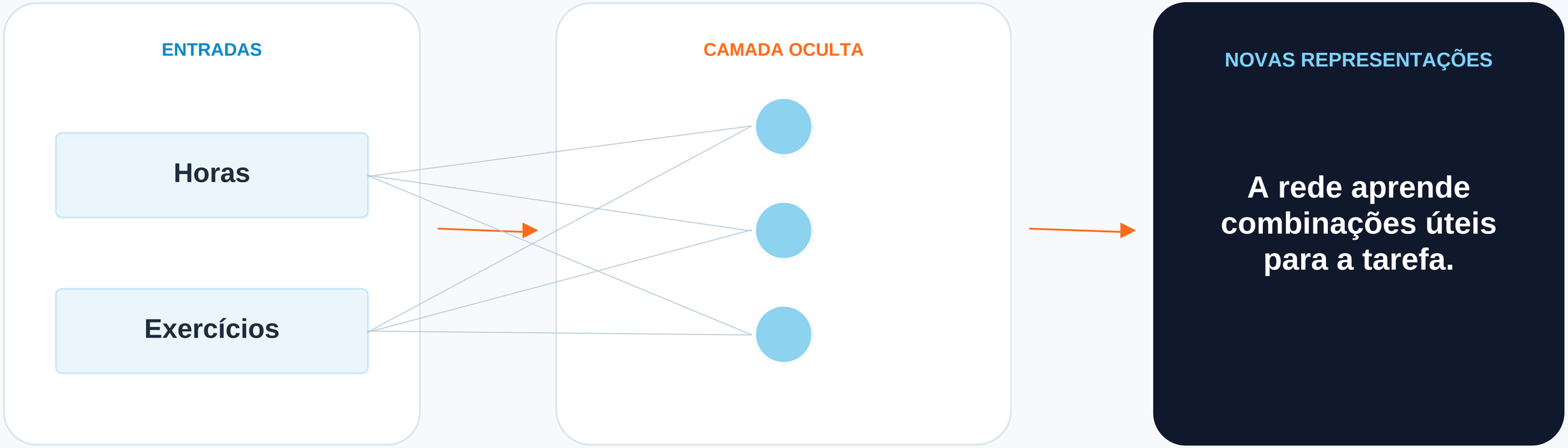
Importante

Os pesos são parâmetros aprendidos durante o treinamento.

Um peso alto em módulo indica contribuição forte; o sinal também importa.

Camadas ocultas: combinar e transformar sinais

O resultado intermediário de uma camada passa a ser a entrada da próxima.



Os valores da camada oculta não são rótulos definidos por nós: são sinais intermediários aprendidos pelo modelo.

Camada de saída: produzir a resposta do modelo

O formato da saída depende do problema que queremos resolver.

Regressão

Um número, como preço estimado ou consumo de combustível.

R$ 350.000

Classificação binária

Um escore usado para decidir entre duas classes.

Sinal → classe 0/1

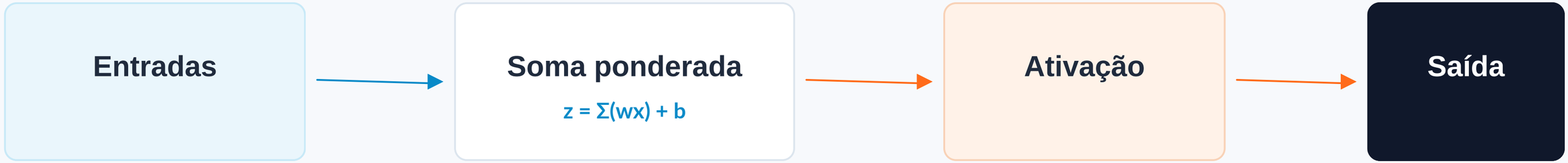
Multiclasse

Um escore para cada classe possível.

[0,2 1,4 0,1]

Função de ativação: uma regra aplicada ao sinal

Primeiro o neurônio combina as entradas. Depois, a ativação transforma o valor calculado.



Pense como um filtro

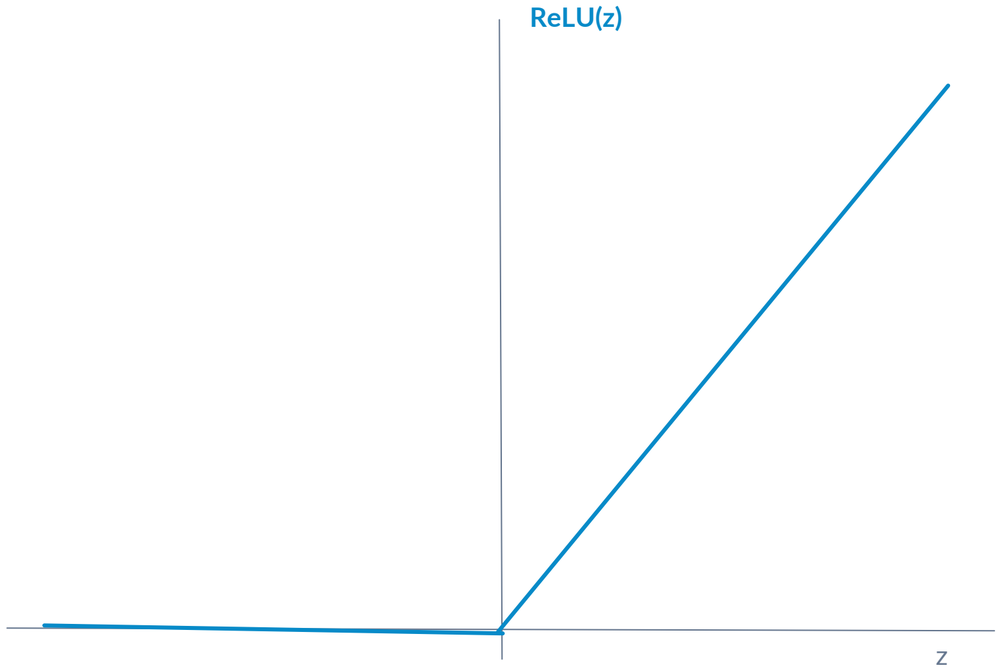
A ativação decide como o neurônio responde ao valor recebido.

Exemplos

Limiar, ReLU e sigmoide são regras diferentes para transformar o mesmo tipo de sinal.

ReLU: z negativo vira zero; z positivo passa

É exemplo de uma função de ativação, muito usada nas camadas ocultas de redes neurais.



ReLU(z) = max(0, z)

z < 0

A saída é 0.

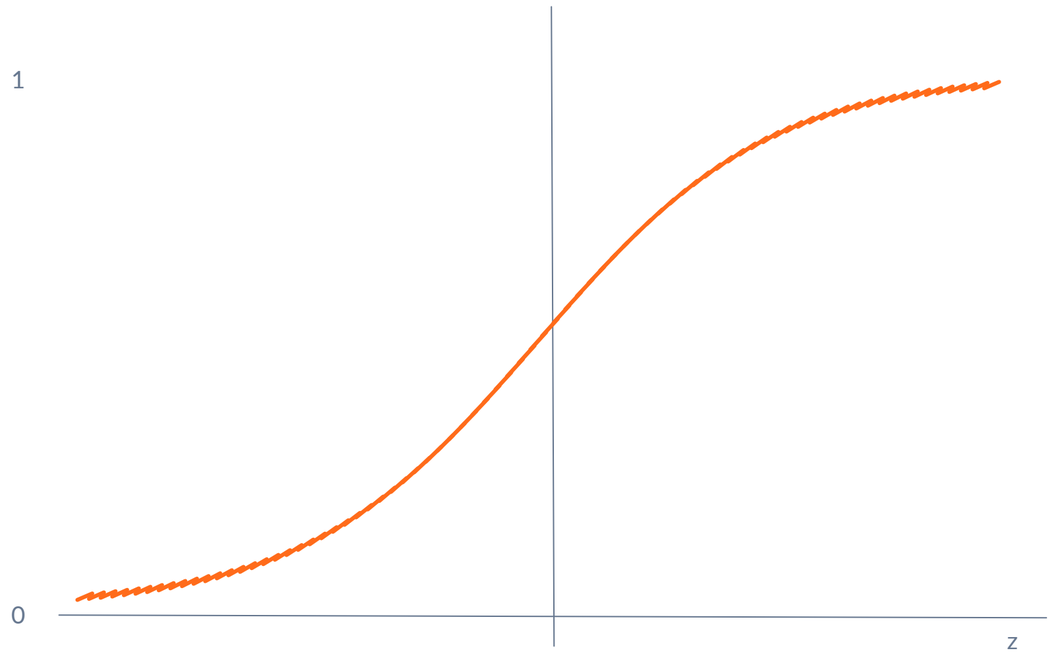
z > 0

A saída é z.

A ReLU não limita os valores positivos a um intervalo de probabilidade.

Sigmoide: transformar um escore em valor entre 0 e 1

Em classificação binária, pode ser usada para interpretar um escore como probabilidade prevista.



$$\sigma(z) = 1 / (1 + e^{-z})$$

Interpretação

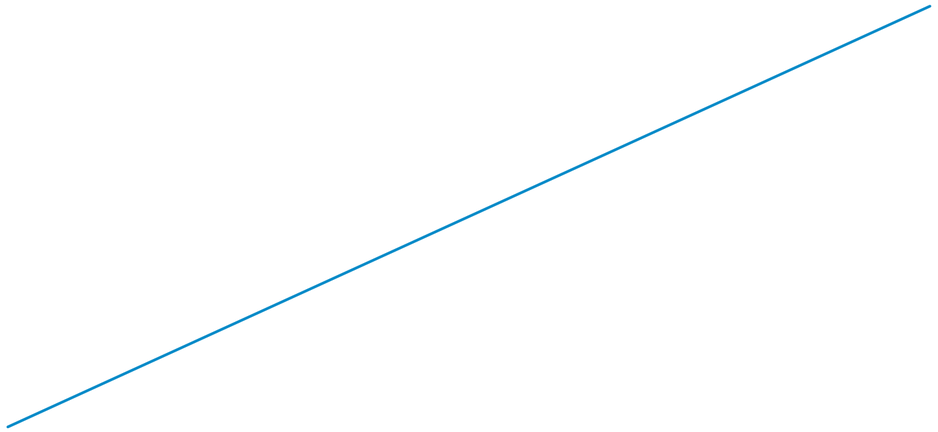
0,90 pode representar uma probabilidade prevista de 90%.

No código, BCEWithLogitsLoss combina o tratamento do escore com a perda; não passe a saída sigmoide a essa função.

Por que a não linearidade importa?

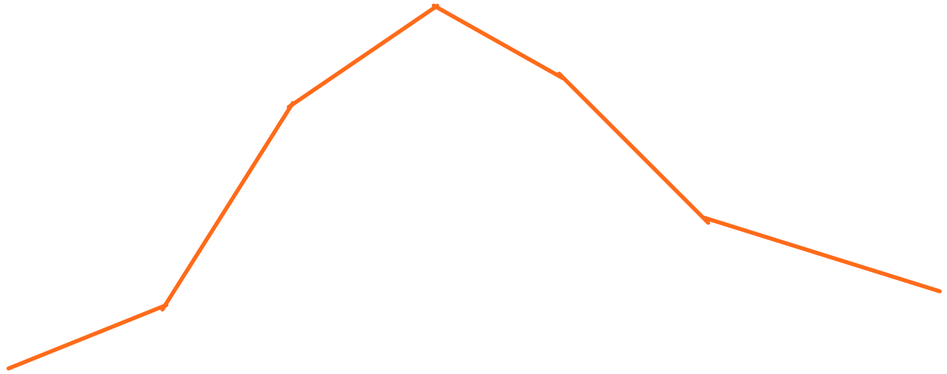
Sem ativação não linear, empilhar camadas lineares não permite representar dados complexos e não lineares

SÓ CAMADAS LINEARES



Uma única fronteira reta

COM ATIVAÇÕES NÃO LINEARES

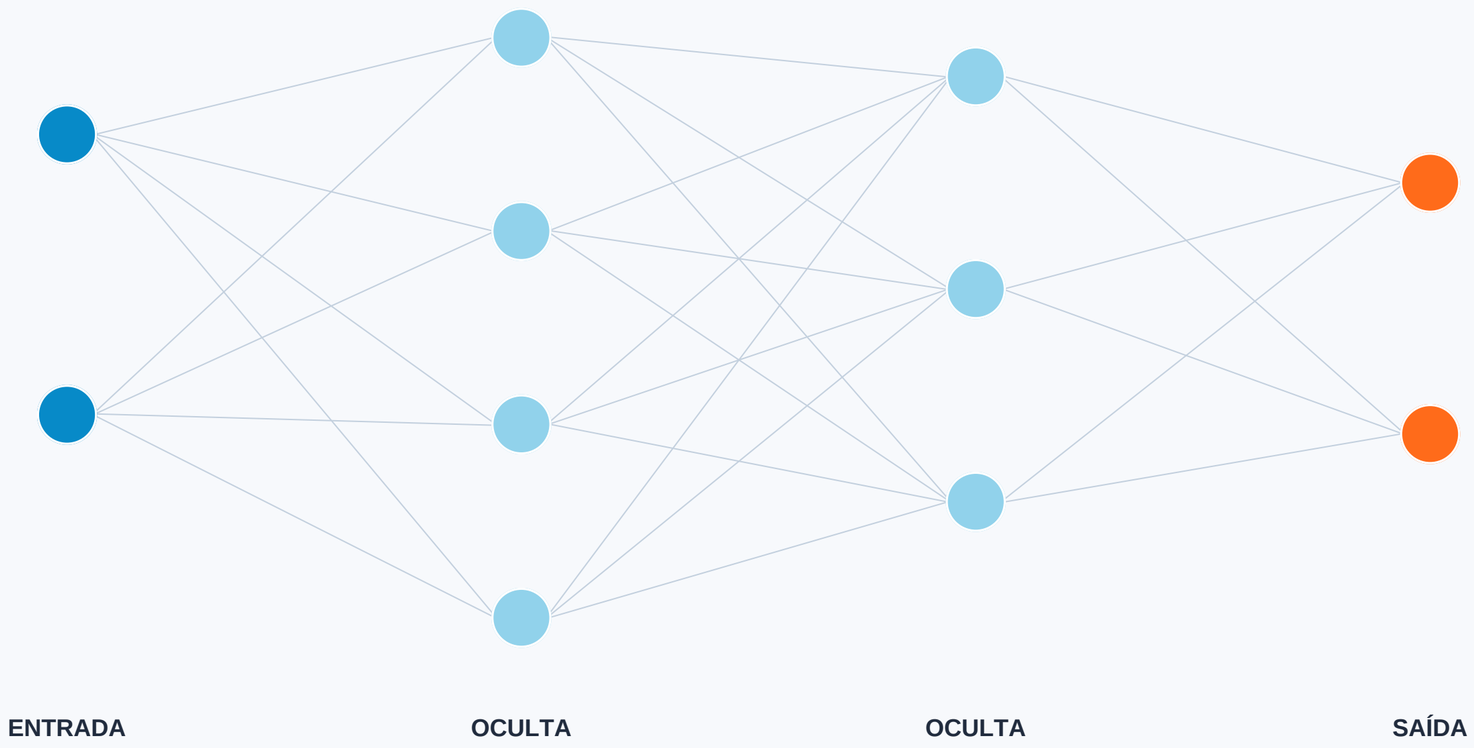


Fronteiras mais flexíveis

A ativação permite que a rede aprenda padrões que não cabem em uma única transformação linear.

Três papéis: entrada, camadas ocultas e saída

Agora que conhecemos os neurônios e a ativação, podemos separar as funções de cada camada.



Entrada

Recebe características.

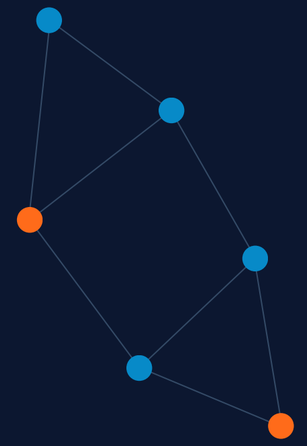
Ocultas

Transformam sinais.

Saída

Produz o resultado.

Do conceito ao treinamento



Vamos acompanhar os dados entrando, a previsão saindo e os parâmetros aprendendo.

TENSOR

Organizar os exemplos

FORWARD + LOSS

Prever e medir o erro

BACKWARD + STEP

Calcular e atualizar

Declarando uma rede pequena

A sequência da arquitetura deixa explícita a ordem em que cada transformação é aplicada.

PYTORCH

```
modelo = nn.Sequential(
    nn.Linear(2, 4),
    nn.ReLU(),
    nn.Linear(4, 1)
)
```

1. nn.Linear(2, 4)

Duas entradas se transformam em quatro valores.

2. nn.ReLU()

Aplica a ativação não linear aos quatro valores.

3. nn.Linear(4, 1)

Combina os quatro valores e gera um escore.

A arquitetura define o caminho de cálculo; os parâmetros ainda precisam ser aprendidos.

Tensores: Representando Dados

Linhas representam instâncias; colunas representam características.

PYTORCH

```
X = torch.tensor([
    [3.0, 4.0],
    [1.0, 2.0],
    [5.0, 6.0]
])

print(X.shape)  # (3, 2)
```

MATRIZ X

3	4
1	2
5	6

3 linhas = 3 exemplos
2 colunas = 2 características

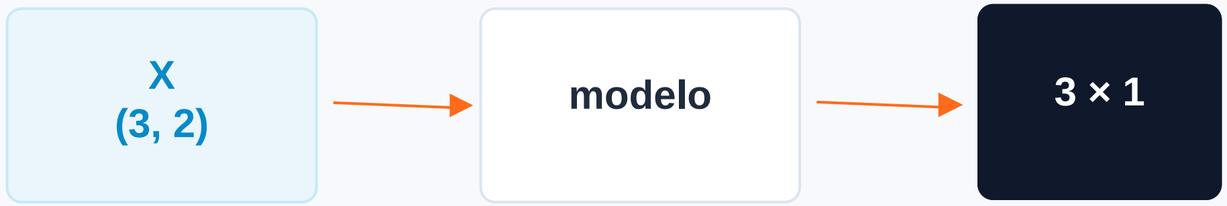
Forward pass: da entrada à previsão

A passada para frente executa as camadas na ordem definida.

PYTORCH

```
logits = modelo(X)

print(logits.shape)  # (3, 1)
print(logits)
```

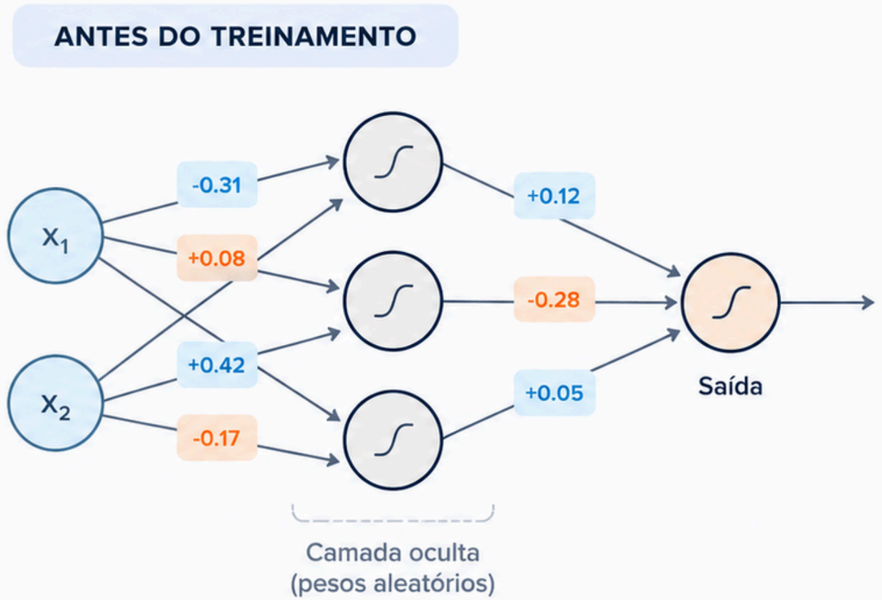


O formato muda

A rede recebe 3 exemplos com 2 características e devolve 3 escores, um por exemplo.

A primeira previsão geralmente é ruim

Os pesos começam inicializados como 0 ou aleatórios, mas ainda não foram ajustados para resolver o problema.





A Função de Perda quantifica o erro

Ela quantifica o quanto as previsões do modelo combinam com os rótulos verdadeiros.

RÓTULO REAL	PREVISÃO
1	0,90

LOSS

```
loss_fn = nn.BCEWithLogitsLoss()
loss = loss_fn(logits, y)
```

BCEWithLogitsLoss recebe o escore bruto (logit), não a probabilidade 0,90 ilustrada à esquerda.

O aprendizado do modelo combina quatro operações

Cada iteração calcula a previsão, mede o erro, calcula gradientes e atualiza os parâmetros.

1 Forward

Calcular a previsão

→

2 Loss

Comparar com o rótulo

→

3 Backward

Calcular gradientes

→

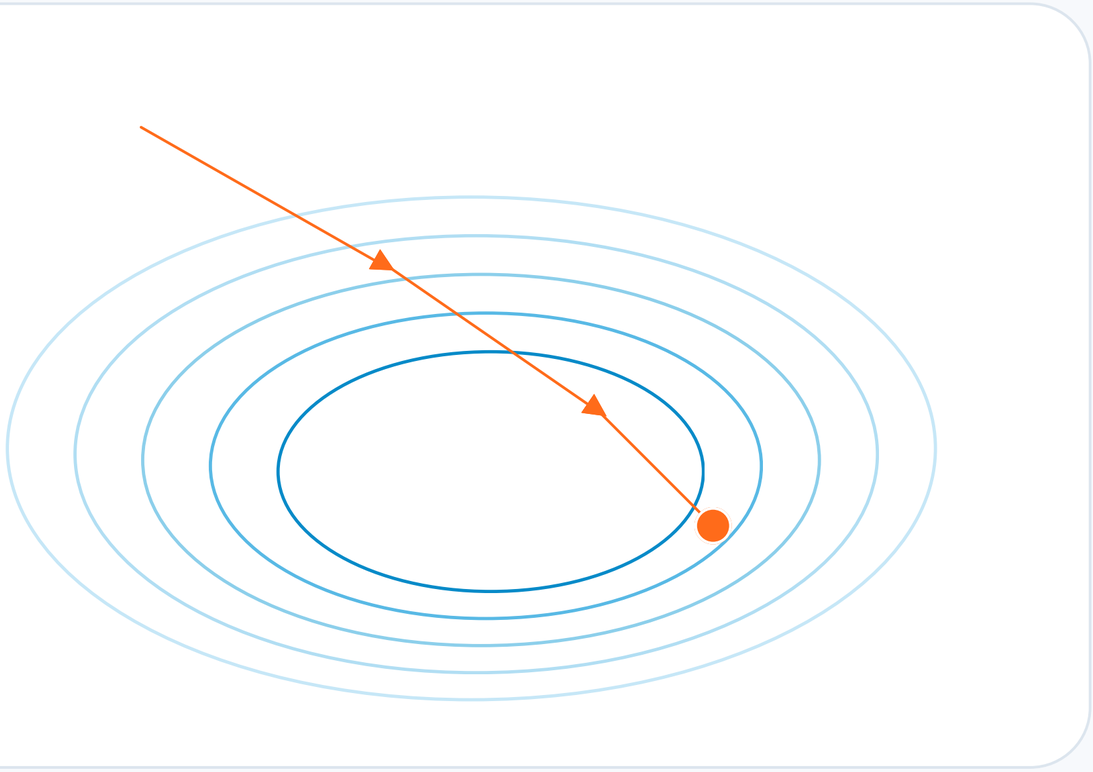
4 Step

Atualizar parâmetros

Repetir o ciclo por várias épocas permite que os parâmetros se ajustem aos dados.

O gradiente indica para onde a perda aumenta

Para reduzir a perda, o otimizador atualiza os parâmetros na direção contrária ao gradiente.



Gradiente

Mostra a direção de maior aumento da função de perda.

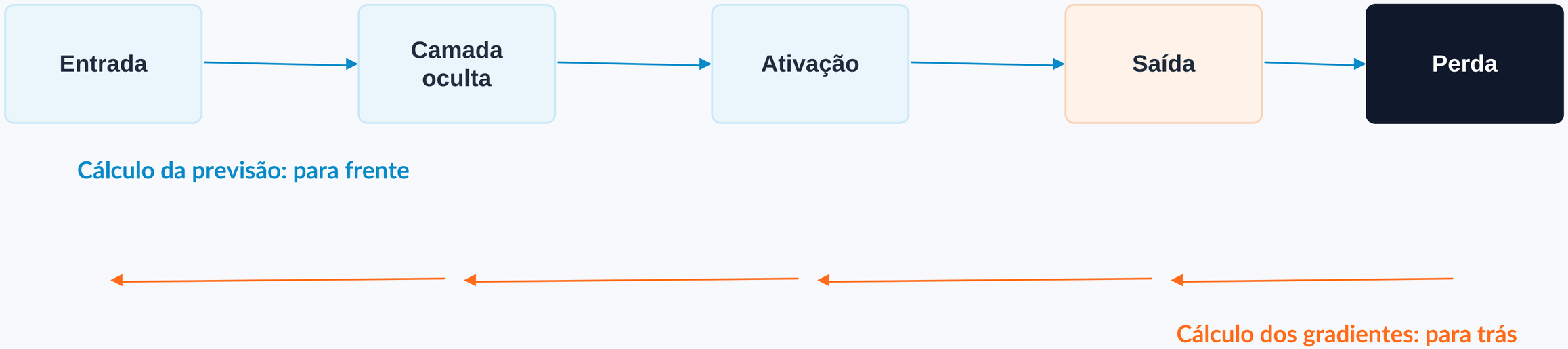
Passo de descida

O otimizador usa o sinal oposto para tentar reduzir a perda.

Taxa de aprendizado muito alta pode fazer os passos ultrapassarem a região desejada; muito baixa torna o treino lento.

Backpropagation: levar o erro de volta pela rede

A retropropagação calcula como a perda muda em relação aos parâmetros de cada camada.



O backpropagation calcula gradientes usando a regra da cadeia; ele não escolhe sozinho o tamanho da atualização.

Otimizador do Modelo - O “aprendizado”

Um calcula os gradientes; o outro usa esses gradientes para mudar os parâmetros.

backward()

Calcula gradientes de perda em relação aos parâmetros e os guarda em `.grad`.

optimizer.step()

Atualiza os parâmetros usando os gradientes calculados e a regra do otimizador.

backward() calcula → step() atualiza

O ciclo do treinamento

Esse é o padrão que você verá repetido em muitos modelos treinados com PyTorch.

PYTORCH

```
for X_batch, y_batch in loader:
    optimizer.zero_grad()
    logits = modelo(X_batch)
    loss = loss_fn(logits, y_batch)
    loss.backward()
    optimizer.step()
```

1. **Limpar gradientes**
   Evita acumular valores da iteração anterior.

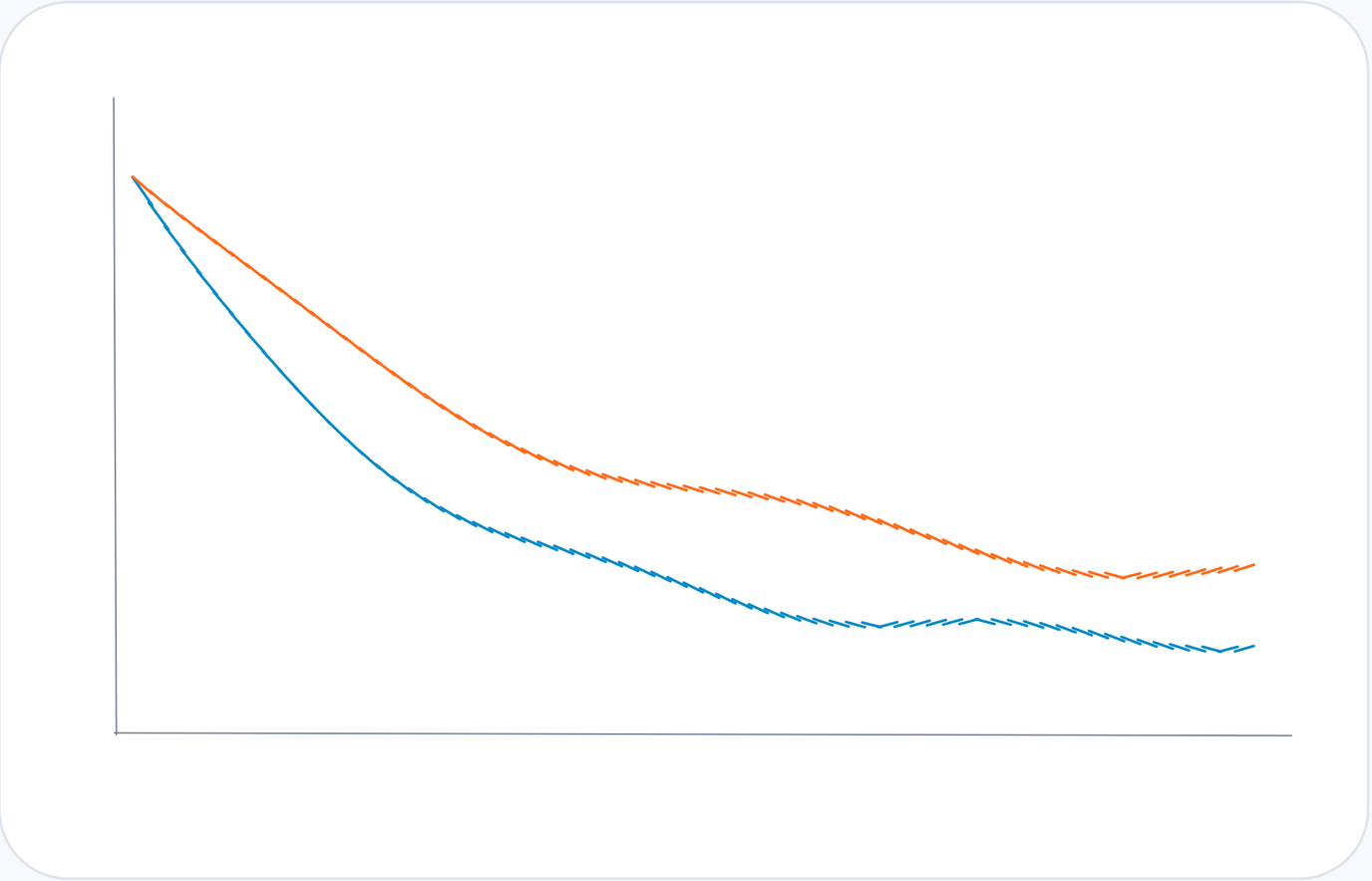
2. **Prever e calcular perda**
   Mede o erro do lote atual.

3. **Calcular gradientes**
   `backward()` propaga o gradiente.

4. **Atualizar parâmetros**
   `step()` faz a atualização.

A curva de perda conta parte da história

Quando a rede aprende padrões presentes nos dados de treino, a perda costuma diminuir.



Perda de treino

Perda de validação

Uma perda menor no treino não prova, sozinha, que o modelo funcionará em dados novos.

Redes neurais também têm dificuldades

A arquitetura não substitui dados adequados, avaliação e cuidado com a tarefa.

Dados e cobertura

Podem falhar em casos pouco representados nos dados de treino.

Sobreajuste (Overfitting)

Podem memorizar exemplos de treino e piorar em dados novos. Comum em redes neurais.

Custo e interpretação

Modelos grandes exigem muitos cálculos e tendem a ser “fechados”: é difícil interpretar os resultados do modelo.

A qualidade de um modelo depende da arquitetura, dos dados, do treinamento e da avaliação.

Que tipos de problema redes neurais podem resolver?

Elas são úteis quando os padrões são complexos e difíceis de escrever como regras manuais.

VISÃO COMPUTACIONAL

Classificar imagens, detectar objetos

ÁUDIO

Reconhecer fala e padrões sonoros

TEXTO

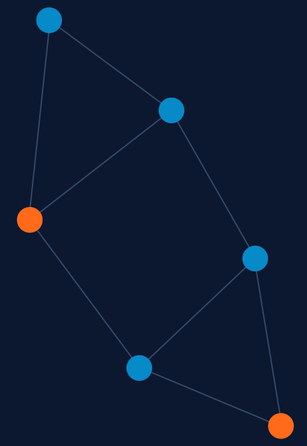
Classificar sentimentos, traduzir, gerar linguagem

SINAIS E PREVISÕES

Modelar relações complexas em dados

Na aula de hoje, focaremos em usar modelos de texto.

Como um modelo trabalha com linguagem



Primeiro representamos o texto numericamente; depois vemos como o contexto pode ajudar.

TEXTO

Tokens e identificadores

VETORES

Representar o conteúdo

CLASSIFICAR

Prever sentimento

O que é processamento de linguagem natural?

É a área que desenvolve métodos para que computadores trabalhem com texto e linguagem humana.

Classificar

spam, sentimento, tema

Traduzir

texto de uma língua para outra

Extrair

nomes, datas, relações

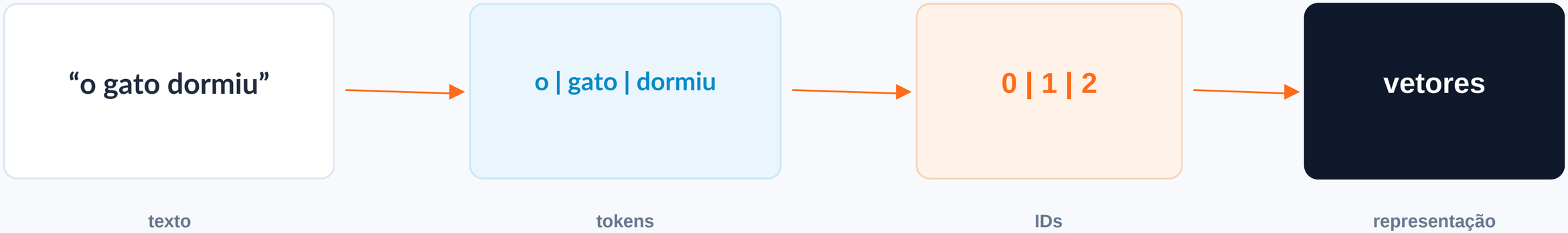
Gerar

respostas, resumos e texto

O primeiro desafio é representar o texto em uma forma numérica que o modelo consiga processar.

O computador precisa de uma representação numérica

Palavras não entram diretamente em uma camada neural: primeiro criamos unidades e representações.



Tokenização

Divide o texto em unidades que o modelo pode identificar.

Vetorização

Converte unidades em números e vetores para a rede processar.

Tokenização: dividir a frase em unidades

Dependendo do tokenizador, uma unidade pode ser uma palavra, parte de uma palavra ou pontuação.

“O banco fechou cedo.”



Um token não é necessariamente uma palavra completa. Modelos modernos também usam subpalavras.

Cada token recebe um identificador inteiro

O índice identifica a unidade dentro do vocabulário; não representa, por si só, o significado da palavra.

VOCABULÁRIO DIDÁTICO

O	0
banco	1
fechou	2
cedo	3

SEQUÊNCIA NUMÉRICA

[0, 1, 2, 3]

Os números são IDs, não notas de importância.

One-hot: um vetor que aponta para um token

Cada token recebe um vetor com um único 1; as demais posições são 0.

VOCABULÁRIO	VETOR ONE-HOT		
gato	1	0	0
cachorro	0	1	0
carro	0	0	1

VOCABULÁRIO COM 3 TOKENS

gato → [1, 0, 0]
cachorro → [0, 1, 0]
carro → [0, 0, 1]

O tamanho do vetor é igual ao tamanho do vocabulário.

O que o one-hot não consegue representar bem

Vetores one-hot identificam tokens, mas não codificam uma relação de semelhança entre eles.

gato	cachorro	carro
[1, 0, 0, 0, 0]	[0, 1, 0, 0, 0]	[0, 0, 1, 0, 0]

Os três vetores continuam igualmente distintos na representação.

Representação esparsa

Quase todas as posições são zero quando o vocabulário é grande.

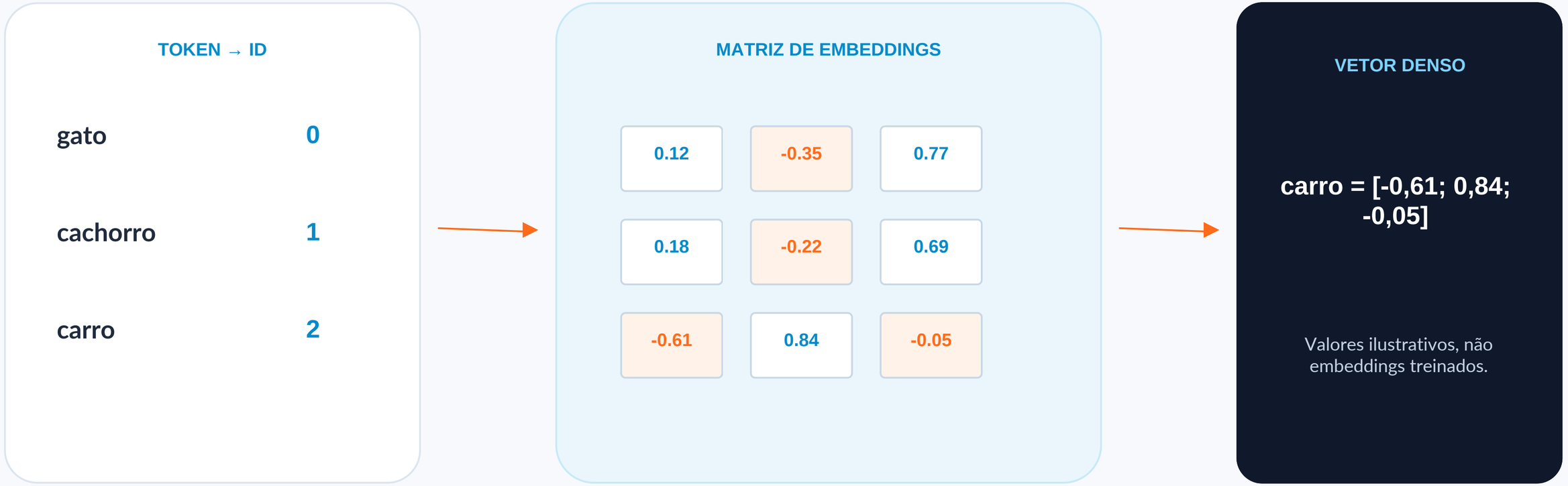
Sem semântica embutida

O one-hot não sabe que gato e cachorro são animais relacionados.

Precisamos de representações mais compactas que possam ser ajustadas durante o treinamento.

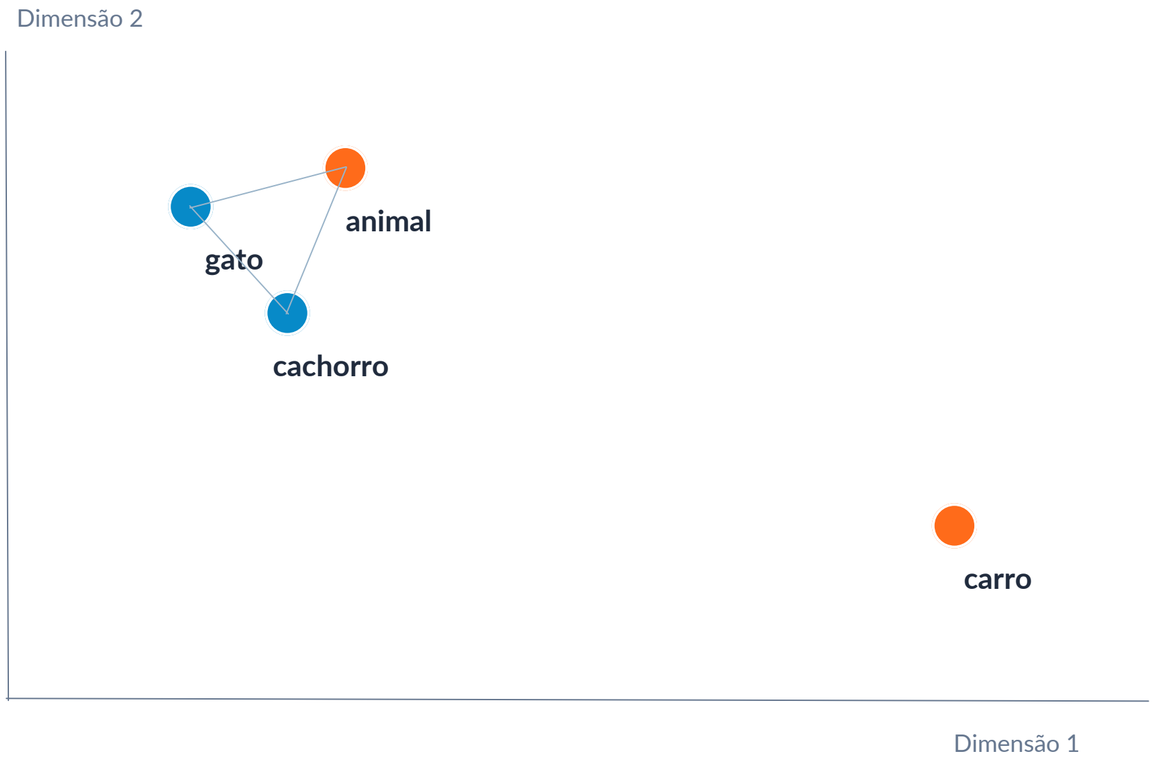
Embeddings: vetores densos para representar tokens

Uma camada de embeddings associa cada ID a um vetor denso (pouquíssimos zeros) de números treináveis.



O que um embedding pode aprender?

Durante o treinamento, vetores podem adquirir propriedades úteis para a tarefa e para o contexto dos dados.



Vetores relacionados

Tokens usados em contextos ou tarefas semelhantes podem ficar próximos.

Não é uma regra fixa

A geometria depende do objetivo e dos dados usados no treinamento.

Esquema ilustrativo em 2D; embeddings reais costumam ter dezenas ou centenas de dimensões.

TF-IDF: transforma texto em um vetor de termos, sem embeddings

Cada coordenada representa um termo; o valor indica sua importância relativa no documento e no corpus.

EXEMPLO DE AVALIAÇÃO

“O filme foi excelente, mas longo.”

filme | excelente | longo

VOCABULÁRIO → VETOR TF-IDF

filme	excelente	longo	outros termos
0,36	0,72	0,29	0

Valores ilustrativos. O vetor real depende do vocabulário e de todo o corpus.

TF • FREQUÊNCIA NO DOCUMENTO

Um termo ganha peso quando aparece no texto.

IDF • RARIDADE NO CORPUS

Termos comuns em quase todos os textos recebem menos peso; termos mais específicos tendem a receber mais.

$$TF\text{-}IDF(t,d) = TF(t,d) \times IDF(t)$$

Resultado: vetor geralmente esparso, com uma coordenada por termo do vocabulário.

Mecanismo Atenção: Como LLMs funcionam?

Em vez de representar cada token isoladamente, o modelo mistura informações de outros tokens relevantes.

FRASE DE EXEMPLO

O | banco | aprovou | o | empréstimo

1 • EMBEDDINGS

Cada token começa com um vetor numérico.

2 • ATENÇÃO

Calcula pesos relativos com outros tokens e combina informação de outros tokens da frase.

3 • REPRESENTAÇÃO CONTEXTUALIZADA

A representação de “banco” incorpora pistas de “aprovou” e “empréstimo”. Em outra frase, outras pistas podem importar.

O QUE O TRANSFORMER FAZ

Repete blocos de atenção e transformação para processar o contexto. Em um LLM, as representações finais ajudam a calcular o próximo token.

Intuição apenas: pesos de atenção não são rótulos de significado nem garantem uma interpretação correta.

Do texto ao classificador: preparar e treinar

Usaremos uma amostra de avaliações rotuladas para treinar uma rede neural pequena em PyTorch.

1 • DADOS + TF-IDF

PYTHON

```python
from datasets import load_dataset
from sklearn.feature_extraction.text import TfidfVectorizer
from torch import nn
from torch.utils.data import DataLoader, TensorDataset
import numpy as np
import torch

imdb = load_dataset("stanfordnlp/imdb")
treino = imdb["train"].shuffle(seed=42).select(range(2500))
teste = imdb["test"].shuffle(seed=42).select(range(800))

vet = TfidfVectorizer(max_features=4000,
    ngram_range=(1, 2), min_df=2)
Xtr_np = vet.fit_transform(treino["text"]).toarray().astype("float32")
Xte_np = vet.transform(teste["text"]).toarray().astype("float32")
```

Ajuste o vocabulário apenas no treino; use o mesmo para transformar o teste.

2 • REDE + TREINAMENTO

PYTORCH

```python
Xtr = torch.from_numpy(Xtr_np)
ytr = torch.tensor(treino["label"], dtype=torch.float32).view(-1, 1)
loader = DataLoader(TensorDataset(Xtr, ytr), batch_size=64, shuffle=True)

modelo = nn.Sequential(
    nn.Linear(Xtr.shape[1], 64), nn.ReLU(),
    nn.Dropout(0.2), nn.Linear(64, 1)
)
loss_fn = nn.BCEWithLogitsLoss()
opt = torch.optim.Adam(modelo.parameters(), lr=1e-3)

for epoca in range(4):
    modelo.train()
    for xb, yb in loader:
        opt.zero_grad()
        loss = loss_fn(modelo(xb), yb)
        loss.backward()
        opt.step()
```

Entrada: vetor TF-IDF • Saída: um logit • Perda: BCEWithLogitsLoss

Avaliar o classificador em avaliações que ele não viu

Depois do treino, medimos os acertos no conjunto de teste e inspecionamos os tipos de erro.

EXECUÇÃO NO CONJUNTO DE TESTE

PYTHON

```
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay
import matplotlib.pyplot as plt
import numpy as np

Xte = torch.from_numpy(Xte_np)
yte = np.asarray(teste["label"], dtype=int)
modelo.eval()
with torch.no_grad():
    probs = torch.sigmoid(modelo(Xte)).squeeze(1).numpy()
preds = (probs >= 0.5).astype(int)

print("Acurácia:", accuracy_score(yte, preds))
ConfusionMatrixDisplay.from_predictions(
    yte, preds, display_labels=["negativo", "positivo"])
plt.show()
```

O número da acurácia e a matriz aparecem quando o notebook é executado; eles dependem do treino e da amostra.

COMO LER O RESULTADO

Acurácia — Proporção de avaliações classificadas corretamente.

Matriz de confusão — Mostra os acertos e se os erros confundem positivos com negativos.

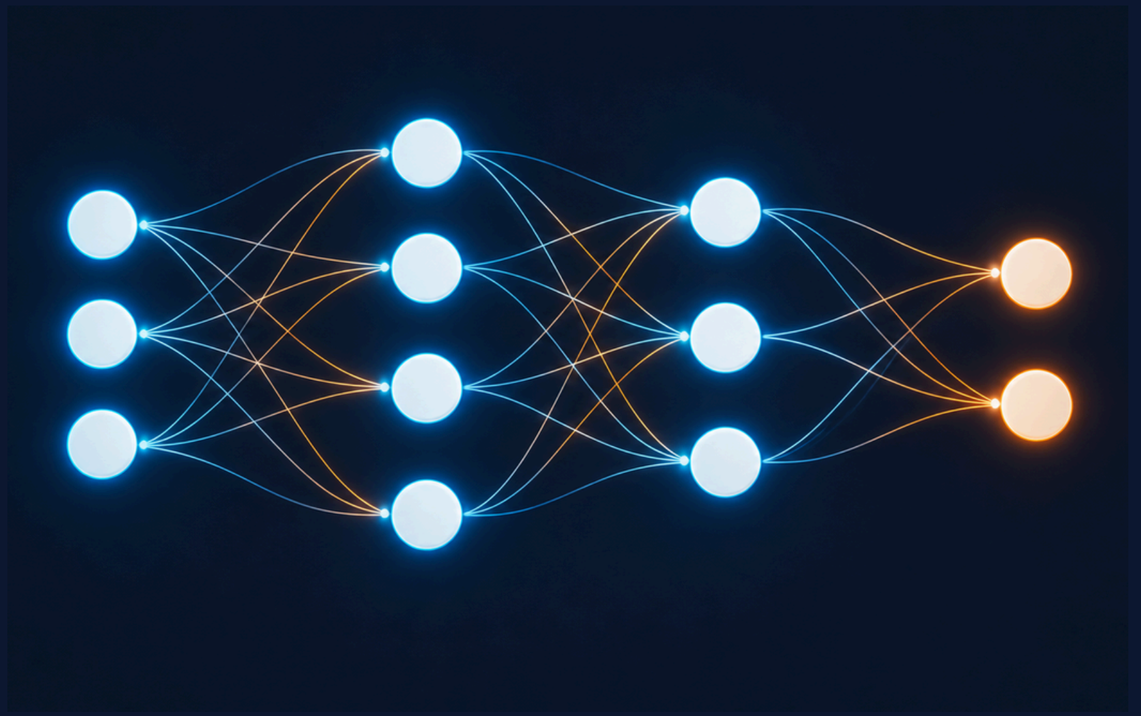
TESTAR UMA NOVA AVALIAÇÃO

Use o mesmo vetorizador para transformar um texto novo; depois, passe o vetor pelo modelo e aplique sigmoid para obter a probabilidade.

Importante: TF-IDF + MLP é um classificador simples, não um Transformer.

Dúvidas?

Uma rede neural aprende a partir dos dados — e o contexto ajuda a representar a linguagem.



https://github.com/GuilhermeHigino05/DataScience-INFOESTE2026